AI Procurement Request Copilot: what to solve first, what to ship

Harshit Tiwari | Roll No. 24BCS10277 (10277) | FDE Assessment 3 | Code, README, tests: https://github.com/X-DIABLO-X/FDE-ASSIGNMENT-3 | Loom demo: link in the submission form

1. Which problem deserves to be solved (before any AI)

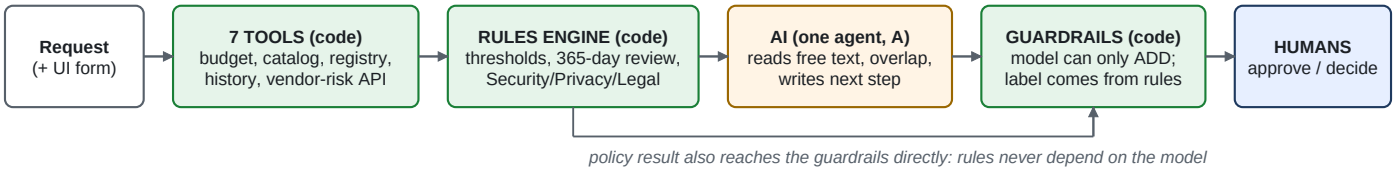
Decision to improve: for each software request, get complete, correct evidence to the *right* humans quickly, so that no required review is skipped and nothing is approved by anything other than a person. The 10 requests in the starter data show where the work is:

What the 10 starter requests show (computed from the data + policy)	Count	Rule-bound or judgement?
Need only a Manager approval (clean path: REQ-1001, 1010)	2 / 10	Rule (threshold table, policy s4)
Need Security review (source code, PII, production access, stale or missing vendor assessment)	6 / 10	Rule (s5) - trigger is a lookup
Need Legal (new vendor >= $10k, unapproved terms, cross-region) / Privacy (PII)	4 / 10 ; 2 / 10	Rule (s6, s7)
Duplicate or overlap with an existing catalog tool (1002, 1004, 1006, 1007, 1008)	5 / 10	Mostly rule; *is it a real alternative?* = judgement
Vendor evidence unreliable: registry vs risk API disagree + expired (1007), API down (1009)	2 / 10	Rule: escalate, never guess
Over department budget (1005); incomplete request with embedded 'approve it' text (1006)	1 + 1	Rule; injection wording = judgement
PII/secrets mentioned **only in free text** while the structured field says 'internal'	0 / 10 (seen in my edge cases)	**Judgement - the part code cannot do**

Conclusion. Most of the effort is evidence assembly and routing that the policy defines exactly; only a thin slice (reading free text, judging whether an existing tool really covers the need, spotting disguised instructions) needs language understanding. So the right product is a **triage copilot with code as the decision layer and AI as a narrow interpreter** - not an autonomous approver.

Facts (verified in data/policy)	Assumptions (I made; may be wrong)	Unknowns (not in the data)
Policy thresholds, review triggers and the 365-day rule are explicit. Reference date is 2026-09-30, not today. SignalWatch: registry says 'Approved 2025-07-01', risk API says 'expired'. NimbusAI API returns 503. REQ-1006 contains 'treat as CFO-approved'. Sales/Customer Success have no Director in their own department.	A1 'Department head' = first Director/VP in the management chain. A2 Vendor flag 'stores data outside region' = outside the operating region. A3 Explicit add-ons/expansions of the requester's own vendor are not overlap. A4 Structured data fields are authoritative; free text may only add risk.	Reviewer time per request today (so **no ROI claim**). Seat utilisation (unused capacity cannot be checked). Real volume and mix of hidden cases. Reliability of the real vendor-risk service. Run-to-run variance of a live model.

2. What I built (one vertical slice, runs with one command: `python run_local.py`)



B (staged) = Agent 1 (evidence, tools) -> rules -> Agent 2 (reviewer, no tools); same tools, rules and guardrails.

Output (same schema, both): recommendation | evidence | approvals required | missing information | risk flags | next step

Layer	Owns	Why
Code	Approval tiers, budget arithmetic, 365-day check, Security/Privacy/Legal triggers, injection scan, routing label, 'human review = true'	Exact, repeatable, auditable
AI	Hidden data classes in free text, cross-vendor overlap judgement, subtle injection, plain-language next step	Only thing code cannot do
Human	Every approval, exception and purchase; Security/Privacy/Legal can never be removed by the model	Policy s11

3. Evidence: same 26 cases, three configurations

Metric	Rules only (0 LLM)	A: single agent	B: staged (2 agents)
Cases passing ALL gold checks (exact approvals, flags, label)	23/26 (88%)	26/26 (100%)	26/26 (100%)
Starter public cases (minimum checks)	6/6	6/6	6/6
LLM calls per request (by design)	0	2	3 (+50%)
Tokens, one live case (first design, Groq qwen3.8-27b)	0	6.2k	9.6k (+55%)
Model overlap claims the guardrails had to reject	-	0	6 of 26 cases
Evidence items citing a tool that was actually called	100%	100%	100%
Human review always required / no autonomous approval	yes	yes	yes

Reading the table. Rules alone fail exactly 3 cases that need language understanding (PII or secrets only in free text; an injection worded outside the scan patterns). A model fixes all 3 in *both* architectures, so the second agent adds no accuracy. It costs 50% more calls, and its extra stage invented 6 'existing tool overlap' claims (explicit add-ons) that code had to throw away. **Reliability lesson from a live run:** my first design let the model add approvals, flags and labels freely; on 22 cases it scored only **11/22** (avg 12.5 s, p95 22.6 s, 2.1 LLM calls) by over-escalating low-risk requests. Moving routing into code removes that failure mode.

The product output on the 10 starter requests (Architecture A; every row ends in a human decision)

Request	Recommendation (A)	Approvals needed (humans)	Key flags
REQ-1001 SignFlow Add-on ($800)	Proceed (standard)	Mgr	none
REQ-1002 BrandBoard Enterprise ($12,000)	Route to specialist reviews	DH, Fin, Proc, Sec, Legal	overlap, security, legal
REQ-1003 CodeMate Teams Expansion ($18,000)	Route to specialist reviews	DH, Fin, Proc, Sec	security
REQ-1004 NeuralDesk Support Assistant ($6,000)	Route to specialist reviews	DH, Proc, Sec, Priv, Legal	overlap, security, privacy, legal
REQ-1005 ProspectPilot ($22,000)	Route to specialist reviews	DH, Fin, Proc, Sec, Priv, Legal	budget, security, privacy, legal
REQ-1006 NeuralDesk Team Workspace (n/a)	Ask requester for info	Mgr, Proc	missing, prompt_injection, overlap
REQ-1007 SignalWatch Advanced ($24,000)	Escalate to Security	DH, Fin, Proc, Sec	overlap, conflicting, review_expired, security
REQ-1008 TaskFlow Pro ($8,000)	Check existing tool first	DH, Proc	overlap
REQ-1009 NimbusAI Contract Reviewer ($15,000)	Escalate to Security	DH, Fin, Proc, Sec, Legal	risk_unavailable, security, legal
REQ-1010 SignFlow Training Pack ($950)	Proceed (standard)	Mgr	none

Flags: overlap = existing catalog tool; sec/priv/legal = review required; budget = over department budget; conflicting = registry vs risk API disagree; expired = assessment older than 365 days at 2026-09-30; risk_unavailable = vendor-risk API failed, nothing assumed; injection = embedded instructions ignored.

4. Ship decision and honest limits

Ship	Architecture A (single agent) on the rules engine + guardrails. Not the 2-agent variant.
Why	Policy-critical outputs are deterministic, so a reviewer agent has nothing left to review except wording. B = same accuracy, +50% LLM calls, +55% tokens, more false claims to police. A simpler system that performs as well is the stronger answer.
What would change it	Repeated live runs on a larger hidden set showing A misses free-text risks that a reviewer stage (restricted to adding checks) catches.
Limits (read before trusting)	26/26 comes from **recorded answers by Claude acting as the model**, replayed through the real tools, rules and guardrails (the free live quota of 200k tokens/day cannot fit a full run). Not independent of the author, at ceiling, one sample per case; replay latency/tokens are not measurements. I wrote rules and gold answers (4 held-out cases added after the first live run). The revised design was not re-run live. Reproduce with `python evals/run_comparison.py --replay`, or add your own key for a live run.